

ComplexGitSync 4.2.2– User Guide

Nicolas Flipó

October 6, 2026

Contents

1	User Guide	2
2	CLI Overview	2
2.1	Prerequisites	2
3	Lifecycle Contract	3
4	Commands	4
4.1	initialise	4
4.2	bootstrap	6
4.3	discover	6
4.4	pull	7
4.5	pull -force	8
4.6	autofix	8
4.7	checkout	9
4.8	branch create and branch list	10
4.9	fetch	10
4.10	branch close	10
4.11	branch check	11
4.12	branch delete	11
4.13	add	11
4.14	rm	11
4.15	commit	12
4.16	push	12
4.17	freeze-release	12
4.18	submodules report and submodules import	13
4.19	submodules init	14
4.20	memory	15
4.21	self-history	18
4.22	verify check and verify repair	20
4.23	env show and env check	21
4.24	view-tree	21
4.25	What cgitsync promises a script	22
4.26	Where a command works: the default workspace	23
4.27	status	23
4.28	merge	25
4.29	pull -private	27

5	Document Formats	27
5.1	Local Authoring Spec (<code>.cgs</code>)	27
5.2	Branches in a <code>.cgs</code>	29
5.2.1	What <code>private</code> means	30
5.2.2	Read-only and writable configuration repositories	30
5.2.3	Which repositories each command touches	31
5.2.4	The two meanings of “force”	32
5.3	Repository Placement and Nested Discovery	32
5.4	Git Tree State Snapshot (<code>.gts</code>)	33
5.5	Local Git Register and Sync Ledger (<code>.lgr</code>)	34
6	Worked Examples: From CGSil1 to cawaqsviz	34
6.1	Level 1 — CGSil1: the minimal synthetic topology	35
6.2	Level 2 — cawaqs: the same minimal style, at scale	35
6.3	Level 3 — cawaqsviz: three ways to the same <code>.cgs</code>	36
6.3.1	3a — Write it by hand	36
6.3.2	3b — Derive it with <code>discover</code> (autodiscover mode)	36
6.3.3	3c — Migrate it with <code>submodules import</code>	37
6.3.4	A shared caveat across all three	38
7	Configuration and Environment	38
8	Error Reference	38
9	Troubleshooting	38

1 User Guide

This chapter is the authoritative reference for every user-facing command, configuration option, document format, and error condition exposed by `ComplexGitSync`.

2 CLI Overview

The single entry-point is the `cgitsync` command.

cgitsync is a Pixi task, not a globally installed executable. This project is developed and run exclusively through Pixi (`pixi install` once, per checkout) — there is no `pip install` that puts a bare `cgitsync` on `$PATH`. Every invocation shown in this chapter as `pixi run cgitsync ...` must be typed exactly that way; a bare `cgitsync ...` will not be found by the shell.

```
$ pixi run cgitsync <command> [options]
```

Running `cgitsync` without a command prints a help summary and exits with code 0.

2.1 Prerequisites

Three tools are needed before the first command: Git clones this checkout and the repositories in a tree; Pixi installs the locked Python environment and runs `cgitsync`; and repository creation needs the provider’s own command line tool — `gh` for GitHub, `glab` for GitLab, or `tea` for Codeberg. Install only the provider tool you use.

The complete list depends on the tree and the work being done:

Dependency	When it is needed	Where its version or requirement comes from
Git	Every clone, inspection and synchronisation.	Its observed version is recorded with the State.
Pixi	Install this project's environment and run <code>cgitsync</code> .	<code>pixi.lock</code> pins the project environment; the observed Pixi version is recorded separately.
Python	Run <code>ComplexGitSync</code> inside the Pixi environment.	<code>pixi.toml</code> selects the interpreter; the observed interpreter is recorded.
<code>gh</code> , <code>glab</code> , or <code>tea</code>	<code>repo create</code> , according to the repository provider.	The provider tool reports its version and whether it is authenticated.
SSH client and agent	Repositories whose remote uses SSH.	Presence is observed. Keys, user names, and key paths are never recorded.
DVC or Git LFS	A tree that uses the corresponding data backend.	The tree declares its use; the observed tool version is recorded when used.
Compilers and system libraries	Native builds or software installed outside Pixi.	The tree declares these in its <code>.cgs [environment]</code> table.
<code>latexmk</code> and TeX	Developers rebuilding the manuals under <code>docs/</code> .	A developer requirement, not an end-user runtime dependency.

The machine must also match a platform supported by the tree's lock files. `pixi run cgitsync env show` reports the operating system, architecture, Pixi platform, libc, tool versions, credential facts, and manifests it observes. `pixi run cgitsync env check` compares that observation with the tree's declared requirements. Neither command installs anything.

`ComplexGitSync` stores no credentials of its own and has no private-repository authentication story: every git operation — `clone`, `pull`, `push`, and the remote lookups behind `checkout`'s branch/tag fallback — runs against whatever the ambient environment already has cached (`ssh-agent`, an HTTPS credential helper, a stored personal-access-token). When that ambient credential is missing or stale, the operation fails immediately with a normal, catchable error rather than sitting silently on a terminal or GUI credential prompt that will never arrive — every git subprocess `ComplexGitSync` runs sets `GIT_TERMINAL_PROMPT=0` and a no-op `GIT_ASKPASS` for exactly this reason. Fix the ambient credential (switch the remote to `ssh`, refresh the token) and re-run the command; nothing needs to be undone first.

3 Lifecycle Contract

This guide follows the simplified canonical lifecycle:

1. `initialise(.cgs)` → clone the dependencies of a checked-out root → **READY** (*nested install*)
`initialise(.gts)` → the same, each repository at the commit the snapshot recorded → **READY**
`bootstrap(.cgs|.gts)` → clone the whole tree, root included → **READY** (*standalone install*)
2. `pull(.cgs|.gts)` → resync an existing tree

3. `checkout(.gts) / add / commit / push / tag` → git operations on the tree
4. `freeze-release` → commit, tag, emit the next `.gts` id and update `<Project_name>.lgr`

The project-local `.lgr` register and sync ledger are both active (T24, T32). Every synchronisation operation is automatically recorded as an immutable DAG event in the `[[ledger]]` section of the `.lgr` file.

4 Commands

Every command answers `-help` with what it does, its options and examples. A group such as `cgitsync memory -help` lists each subcommand with its options, and `cgitsync help -all` prints every command and option on one page, ready for `grep`. `help` is a command of its own: `cgitsync help memory explore` is the same as `cgitsync memory explore -help`. The help is generated from the parser the command actually runs, so it does not drift from it.

Every command follows one grammar. A subcommand is a plain word (`branch close`, `verify repair`); anything that starts with `-` is an option and never a command of its own; a single-dash `-x` is only ever the short form of a `-option`. An old spelling such as `close-branch` is not run: `cgitsync` names its new form (`'close-branch'` is now `'branch close'`.) and stops.

The CLI exposes three command groups once a `GitTree` is `READY`. The minimalist level is `initialise`, `bootstrap`, `freeze-release`, `status`, and `view-tree`. The expert level exposes the individual operations: `branch create`, `branch list`, `branch close`, `branch check`, `branch delete`, `checkout`, `validate`, `pull`, `pull -force`, `fetch`, `autofix`, `add`, `commit`, `push`, `tag`, `submodules report`, `submodules import`, `submodules init`, `verify check` and `verify repair`, plus `env show` and `env check`. The configuration level, `discover` and `repo`, authors a `.cgs` specification without requiring existing state. There is no command that deletes a workspace for you: to start a directory over, move it aside yourself. Redundant inspection aliases such as `print`, `view-operation`, and `validate-topology` are not part of the public CLI surface.

4.1 initialise

```
$ pixi run cgitsync initialise <spec.cgs> [--output-path DIR] [--commit-
  gitignore]
  [--git-user-name NAME] [--git-user-email EMAIL]
  [--force-protocol {ssh,https}]
$ pixi run cgitsync initialise <snapshot.gts>
$ pixi run cgitsync initialise --project NAME --repo PROVIDER:OWNER/REPO
  [--repo ...] [--output-path DIR]
```

The *nested install*. `initialise` builds the dependencies of a project whose root is already checked out at `CGSHOME`, with the running `cgitsync` inside it. It refuses — before touching the disk, naming `bootstrap` — when `CGSHOME` is not a Git checkout (a detached `HEAD` is fine) or the running installation is not inside it. A `.gts` source checks each dependency out at the commit the snapshot recorded, and is refused, naming every repository, when a remote no longer holds one. A `private`, `writable` entry clones the branch computed from the project's name, then its `fallback_branch`, then the remote's own active branch. Dispatches on file extension:

- `.cgs` source (clone mode): clones the workspace's dependencies. The optional `-output-path` controls `CGSPATH`; `CGSHOME` is derived as `CGSPATH/<project-name>`. When omitted, `CGSPATH` defaults to `../...` On success, ends in `READY`.

- `.gts` source (restore mode): clones the same dependencies and puts each on the commit the snapshot recorded, so the rebuilt tree carries the snapshot's State name. On success, ends in `READY`.
- Direct CLI authoring: requires `-project` and at least one repeatable `-repo`. It is mutually exclusive with a source file. The CLI collects values only and calls `ComplexGitSyncClient.configure()` (the `.cgs` writer, which has no command of its own); that reusable Python API delegates parsing, normalization, and validation to `cgs_format.py` and produces the same canonical `CgsDocument` as an equivalent `.cgs` file.

All initialization paths end in a `READY` tree or fail explicitly.

initialise re-clones your dependencies. Only the root repository at `CGSHOME` is kept as it is. Every repository below it whose directory already holds files is **deleted and cloned again**. The old `.git` goes with it, so nothing in that directory can be recovered once the run has passed that point.

Before deleting anything, `initialise` inspects each destination and stops the whole run when one of them holds work that exists nowhere else: uncommitted changes, commits that have not been pushed to the branch's upstream, or a branch with no upstream at all. It names every repository that blocked it, in one message, and deletes none of them. No flag overrides this. The message says what to do: commit and push, or move those directories aside yourself.

A destination that is not a Git checkout at all — what a clone interrupted halfway leaves behind — is still cleared without any flag. That is the case the deletion was written for, and it still works.

Before a `.cgs` source is confirmed ready, every repository with children (root, or any nested repository that itself has further nested children) is safely pulled parent-first and has its `.gitignore` updated with the relative path of each immediate child — nested repositories are plain independent clones rather than gitlinks, so without this a parent's `git status/git add` would otherwise see a child's working tree as ordinary untracked content. This is logged as the `GT-GITIGNORE` phase, after `GT-CLONE`. If the safe pull for one of these repositories fails, `initialise` raises immediately and nothing is written; run `pull -force` on that repository, then `initialise` again. By default nothing is staged, committed, or pushed by this step — it only writes the file and prints what changed. Pass `-commit-gitignore` to explicitly approve staging (`.gitignore` alone), committing (with a message listing exactly which children were added), and pushing each changed repository. The commit identity defaults to local git config; `-git-user-name/-git-user-email` override it and persist the override to `CGSHOME/.cgitsync/master.toml` via `MasterConfig`, so later invocations on the same workspace pick it up without repeating the flags.

The root's own `.gitignore` additionally always gains `.cgitsync/` (the generated runtime-state directory — snapshots, ledger, run logs, never part of the `.cgs/.gts` project spec; the run logs never leave the machine, are never pushed with a memory, and only the 200 most recent are kept) and its `<project-name>.lgr` register file, whether or not the tree has any nested repositories at all — that state is written under the root regardless of tree shape, so it is excluded the same idempotent way, via the same underlying write `sync_gitignore()` already uses for child mount points.

`-force-protocol {ssh,https}` is an expert option, typically used systematically in CI rather than for everyday development. It overrides `access_protocol` in memory for every repository this run clones — including ones discovered later from a nested `.cgs` living in a different, separately-cloned repository — so no `.cgs` file is ever read differently or written to. Forcing `https` is the usual CI case: it removes the dependency on an SSH key or agent being present on the runner, a dependency that otherwise differs between a `push` run and a `pull_request` run (GitHub Actions withholds repository secrets from a pull request opened from a fork). Omitted

(the default), each repository follows whatever protocol its own `.cgs` entry declares, exactly as without the flag.

4.2 bootstrap

```
$ pixi run cgitssync bootstrap <spec.cgs|snapshot.gts> <project_name> [--  
  cgs-path DIR]  
  [--force-protocol {ssh,https}]
```

The *standalone install*: clones the full repository tree from a `.cgs` or `.gts` source (root included) into an isolated CGSHOME, for running `cgitssync` from its own standalone clone rather than nested inside the project tree it manages — one `ComplexGitSync` clone reused across many projects. It delegates to the same `ComplexGitSyncClient.clone_cgs` pipeline, which picks each repository’s branch by the fallback chain and discovers nested `.cgs` files after the parents are cloned, but differs from `initialise` in two ways:

- `project_name` is a required positional argument, not inferred from the `.cgs` document. It always forms the final path segment of CGSHOME, regardless of the specification’s own project name.
- CGSPATH does not default to `../..` relative to the current directory (that default assumes `ComplexGitSync` is cloned inside the tree it manages — the nested-mode case `initialise` is for). When `-cgs-path` is omitted, CGSPATH instead defaults to a fresh `$HOME/.cgs/CGS<timestamp>/` directory, created automatically if it does not exist, so a bootstrapped project can never land inside `ComplexGitSync`’s own clone.

`-force-protocol {ssh,https}` behaves exactly as described under `initialise` above, applying here to every repository the run clones — including the root, which `bootstrap` (unlike `initialise`) also clones from scratch rather than assuming already checked out.

4.3 discover

```
$ pixi run cgitssync discover [ROOT] [--write FILE] [--max-depth N]
```

Scans `ROOT` (default: the current directory) for git repositories and drafts a `.cgs` from what is checked out, through the `ComplexGitSyncClient.discover_repos()` facade. This is the entry point for adopting a project that exists on disk but has no `.cgs` describing it yet.

The command is a pure read of the filesystem and of each repository’s git config: nothing is cloned, fetched, staged, or modified, and no network call is made. Without `-write` it only prints what it found, matching the “report first, write only when asked” posture of `-commit-gitignore` and of `submodules report` before `submodules import`.

The walk descends with no depth limit by default, treating every directory that contains a `.git` entry as a repository. Pass `-max-depth N` (`ROOT` itself is depth 0) to bound it. It never descends *into* a `.git` directory: for a submodule the real git directory lives at `<parent>/.git/modules/<name>` while the child’s own `.git` is a file, so walking into it would report the same repository twice.

For each repository, `origin`’s URL is read and converted to the canonical `provider:owner/repository` shorthand through the existing `parse_repo_id()` grammar — this command adds no second parser. `relative_path` is taken straight from the walk rather than from a repository name, so a child mounted at `external/Thing` is recorded there and not at `Thing`. `nested_config` is left unset on every entry: the default `auto` already resolves cleanly whether or not a repository carries its own `*.cgs`.

Repositories with no `origin`, or whose remote URL does not resolve to a registered provider, are reported as warnings and left out of the draft. They are never guessed at: a draft that

silently invented an address would be worse than one that states what it could not determine. The warning names what to add, not only what was missing — a ready-to-edit entry with the address it could not fill in. Always review the generated file, then **validate** it.

The branch each repository is actually checked out on is drafted twice, for two different jobs. The root repository’s own branch becomes **project.default_branch** — the tree’s target, read back the same way any other declared branch is (“Branches in a **.cgs**”, below). Every repository also keeps its scanned branch as **fallback_branch**, consulted only when the target branch turns out to be absent from the remote. A repository scanned on a *different* branch than the root gets its own explicit **default_branch** too, so it is not silently retargeted at the root’s branch on the next **initialise**; one scanned on the same branch as the root needs no per-entry override at all. Re-materialising the drafted tree therefore reproduces the tree that was scanned, branch included — before this, the branch was drafted only as a fallback, which **initialise** never targets first, so a tree scanned entirely on a non-**main** branch quietly re-cloned onto **main** wherever the remote happened to have it.

A repository found *inside* another repository is drafted as that repository’s child, not the project root’s. Nesting is worked out by comparing paths: every path in a **.cgs** is written from the project root, so a repository whose path lies under another’s sits inside it, and the innermost one wins. The report marks such an entry **inside**: **<path>** and prints the resulting tree, and **build_registry_from_cgs_document()** reads the drafted file back the same way — the holding repository becomes a **PARENT** and keeps its own **.gitignore** entry for the repository it holds, which is what stops a later **add** from restaging that repository as a submodule.

Only what is *checked out at scan time* can be found. A repository cloned without **-recurse-submodules** leaves its submodule paths as empty directories, and those are correctly not reported; recovering them from git metadata instead is the job of **submodules report** and **submodules import**. The same applies when **-max-depth** is given: if it stops the walk while directories are still left to look into, the report says so in a warning rather than presenting a partial answer as a complete one. Without the flag, the walk cannot stop early — there is nothing to stop it at.

4.4 pull

```
$ pixi run cgitssync pull [spec.cgs|snapshot.gts] [--commit-gitignore]
  [--git-user-name NAME] [--git-user-email EMAIL]
  [--force-protocol {ssh,https}] [--private]
```

Resynchronises the project tree. **.cgs** sources reload the source, discover nested configs, then resynchronise the loaded tree. **.gts** sources load the saved registry as the starting point and then run the same pull workflow. In both cases the operation starts at the project root: **root -> parent -> leaf**. Every repository — root, parent, and leaf alike — is a plain independent clone and receives its own **git pull -ff-only**. Use **checkout <tag> -ref-kind tag** when you want to check out a frozen recorded release state instead of pulling current remotes. If local changes block this safe pull, the CLI suggests **cgitssync autofix**, and names any repository whose commits only exist locally, because **pull -force** refuses while they do.

Before pulling your own branch, **pull** fetches *every* branch of each remote. That is what lets **checkout** join a branch somebody else pushed rather than starting a new one of the same name (see **checkout** below). It also repairs a workspace whose clone left a single-branch fetch refspec behind: a clone downloads one branch, and a workspace cloned by an older build may still carry that narrow refspec.

For a **.cgs** source, **pull** also runs the same **.gitignore** lifecycle sync described under **initialise** — including **-commit-gitignore** and the **-git-user-name/-git-user-email** identity override — once the tree-wide pull completes. A **.gts** source runs no discovery, so there is nothing new for the sync to find. **pull -force** does not run this sync at all, and so does not

take `-commit-gitignore`, `-git-user-name` or `-git-user-email`; it is a destructive recovery command, not a lifecycle path the sync is wired into.

`-force-protocol {ssh,https}` rewrites every repo's remote to that protocol before pulling, persisting the change (`git remote set-url`, a no-op when the URL already matches) rather than a one-off override — the switch sticks for every command after this one too, the same way a repo's protocol at clone time sticks for everything downstream of it. Same meaning as `initialise/bootstrap's -force-protocol`, applied to an already-cloned tree instead of at clone time — useful when a repo was adopted rather than cloned by `ComplexGitSync` (`initialise` never touches an adopted root's remote) and its remote's protocol does not match what the ambient environment can actually authenticate with. On a failure that looks like an SSH or HTTPS auth problem, the error gains an actionable hint naming `-force-protocol <the other one>` even without the flag.

The rewrite reads the repository's *actual current* remote URL and only swaps its scheme — `ssh (user@host:path)` to/from `https/http (scheme://host/path)` — keeping the host and path exactly as they already are. It does *not* rebuild the URL from the repository's stored identity fields (`gitprovider/project_owner_name/...`): those can be missing or stale for a repository loaded from an older `.gts` snapshot (see the “Formal `.gts` snapshot specification” section of the architecture reference), and rebuilding from a wrong or default-GitHub provider would silently aim the pull at the wrong host. Converting the URL that is already configured never has that failure mode. A remote in neither recognised form (a bare local filesystem path, for instance) cannot have its protocol converted; `-force-protocol` then fails with an explicit error naming the repository, rather than guessing.

4.5 pull -force

```
$ pixi run cgitssync pull --force [spec.cgs|snapshot.gts] [--force-protocol {ssh,https}]
  [--private]
```

Destructively resynchronises the same tree. Every repository — root, parent, and leaf alike — is forced to the selected remote branch, parent-first, with `git fetch`, `git checkout -B <branch> FETCH_HEAD`, and `git clean -fd`. Uncommitted changes and untracked files are not discarded: each repository's is set aside first with `git stash push -u`, a warning names it, and `git stash pop` brings the work back. The command never force-pushes, and never leaves a commit on no branch: if any repository holds commits that no remote has, it refuses for the whole tree before touching anything, names the repositories, and asks you to push or merge first. `-force-protocol` behaves exactly as described under `pull` above.

4.6 autofix

```
$ pixi run cgitssync autofix [spec.cgs|snapshot.gts] [--error TEXT] [--repo NAME]
```

Diagnoses and repairs the situation named by the last failing command's error, instead of a person reading raw `git` output and guessing which of `merge/pull -force` is safe. With no `-error`, reads the most recent `.cgitssync/logs/*.log`'s failing command (run logs stay local in that folder, so a `push` that folded the memory never moves the log `autofix` needs) — the normal workflow is a command fails, run `cgitssync autofix` next, and nothing gets retyped. `-repo` names which mounted repository to repair when it cannot be guessed from the error text (a chain-shaped repository's name is usually visible in its own remote URL, which `git`'s error text already includes).

Tricky git states, and the safe `cgitsync` command for each. The SYNC column status prints (§4.27, below) names the git-level situation; it cannot say whether the content behind it is safe to reconcile automatically. This table can:

SYNC state	Safe
ahead(+N)	push
behind(-N)	behind
diverged, plain-text content (e.g. <code>.localSpec</code> , <code>.claude</code>)	pull ply merg
diverged, chain-shaped content (<code>.memory</code> 's <code>lgr/</code> ledger, and — once built — <code>omniscience</code> 's journal)	way cont files auto hold an a noth drop Refu a gu
Anything <code>autofix</code> does not recognise	

`autofix` tells a plain-text divergence apart from a chain-shaped one before acting on either: fetches, attempts the ordinary merge, and only on a conflict in a repository registered as chain-shaped does it re-sequence both sides' new ledger entries — in the order they were actually recorded, not “whichever side pushed first” — verify the result, and complete a real two-parent merge commit. It never force-pushes and never rewrites history; a repair it cannot verify as correct is refused, leaving the conflict exactly as `git` left it, rather than writing something that looks resolved and is not.

4.7 checkout

```
$ pixi run cgitsync checkout <branch> [--gts <snapshot.gts>] [--ref-kind  
branch|tag]
```

Checks out a branch or tag across the whole tree, parent-first. Loads the READY registry from `-gts`, then propagates the ref, creates missing local branches, and runs `git checkout` in every repo. On success the tree remains READY and a new `.gts` snapshot is written.

A branch that exists on the remote is joined, not recreated. When the repository already holds a remote-tracking ref for the branch — because some earlier `pull` downloaded it, or another `checkout/branch create` fetched it on demand — `cgitsync` creates the local branch *from that ref*, with tracking set. You land on your colleague's commits. When it has no such ref cached, it asks the remote directly (one `ls-remote`, no full fetch) before deciding: found, it fetches that one branch and joins it the same way; genuinely not found, it creates a new branch where you currently stand, at no extra cost beyond that one lookup.

This is a change of behaviour, and the reason for it is the failure it removes. `cgitsync` used to create the branch at the current HEAD whenever this workspace had not already cached the remote's ref — so typing a colleague's branch name, on a clone that had never fetched it, gave you a branch of that name holding commits that had nothing to do with theirs, and nothing in the output said so. Depending on some earlier command (a `pull`, or a differently-built `cgitsync` during `bootstrap`) to have fetched every branch first was the actual defect: one command's gap silently became another command's fork.

pull still fetches every branch of each remote before pulling your own, and remains the cheapest way to bring many refs across at once. **checkout** and **branch create** no longer depend on it having run first: each fetches the one ref it needs, exactly when it needs it.

4.8 branch create and branch list

```
$ pixi run cgitssync branch create <branch> [--private] [--gts <snapshot.gts>]
$ pixi run cgitssync branch list [--per-repo] [--private] [--gts <snapshot.gts>]
```

branch create creates a branch across the whole **READY** tree without checking it out. **branch list** prints the project's own branches: those of the root repository, local or on origin as of the last fetch, the one the project is on marked with *. Each line says which repositories hold the branch they would be on under it, or names those that lack it; a **closed/** branch is listed apart. **-per-repo** prints one line per repository with its own local branches instead. Changes nothing. On either subcommand, **-private** narrows it to the writable configuration repositories.

4.9 fetch

```
$ pixi run cgitssync fetch [--private] [--gts <snapshot.gts>]
```

Runs **git fetch -prune origin** in every cloned repository of the tree, parent-first. Only the remote-tracking refs change: no local branch, no **HEAD**, no **worktree**, and no **State** is written. A branch pushed elsewhere appears in **branch list** afterwards, and one deleted on origin disappears. Prints **fetches** or **skipped** (with the reason: not cloned, no origin, or the fetch failed) for each repository, then a summary; exits non-zero if any fetch failed. **-private** narrows it to the writable configuration repositories.

4.10 branch close

```
$ pixi run cgitssync branch close <branch> [--private] [--gts <snapshot.gts>]
```

Closes a *project* branch: keeps what it alone holds on the project's **ancestors** branch, then renames it to **closed/<branch>** tree-wide, leaf-first, locally and on the remote. Each repository closes the branch it follows under that project branch: the same name in a project repository, **<project>_<branch>** in a private/local one (so **branch close tmpPyPi** closes **ComplexGitSync_tmpPyPi** in **.memory**), and nothing in a private/distant one, which never follows the project and is reported as skipped. Requires **READY**; the tree remains **READY** on success. Renames only: nothing is deleted, so the commits stay exactly as reachable as before, under a name that says what happened to them. Per repository, the branch is pushed under its closed name first, never forced; the old remote name is removed only once the remote is seen to hold the closed one; only then is the local branch renamed. A branch that exists only on the remote, never checked out here (run **cgitssync fetch** first to see it), is closed there from its remote-tracking ref. A branch the remote never had is renamed locally only.

Refuses, before touching any repository, in two cases: closing the project's own default branch (every repository with no branch of its own falls back to it), and closing a branch that any repository in scope is currently checked out on, which it names, and a branch that lacks commits its remote holds, which would be lost (pull or merge it first). This command closes a branch, it does not move the tree off one; check it out elsewhere first. A repository with no such branch is skipped, not an error.

-private selects the writable configuration repositories instead of the project's own, the same as **branch create**.

Closing is the safe point. Before anything is renamed, every commit the branch alone holds — one no other branch, local or on origin, reaches — is kept on the project’s permanent **ancestors** branch: **ancestors** in a project repository, `<project>_ancestors` in a private/local one. It starts as an empty commit and gains one merge per branch it keeps; the merge keeps **ancestors**’ own (empty) tree and takes the branch as its second parent, so its whole history stays reachable. Nothing is rewritten and nothing is forced: **ancestors** only ever gains a commit. Each move is recorded in the ledger (the entry’s **relocations**: what moved, from which address, to which, and its hash there), and the ledger is checked to still verify. A step that fails refuses with nothing renamed. Afterwards the closed branch can be deleted by anything, **cgitsync** or plain Git or the provider’s web page, without losing a commit. **ancestors** itself is never closed or deleted. Run on a branch that is already closed — one closed before **ancestors** existed — **branch close** keeps and records what it alone holds and renames nothing.

4.11 branch check

```
$ pixi run cgitsync branch check <branch> [--private] [--gts <snapshot.gts>]
```

Says, per repository, what deleting `<branch>` would lose: the commits only it reaches, and the ledger entries and State files only it holds (by content, so a copy on another branch counts as kept). Each repository is **safe** (nothing would be lost), **recorded** (**ancestors** keeps it and the ledger records the move) or **needs ancestor**. `<branch>` may be a live branch, `closed/<branch>`, or a closed branch by its old name. It first fetches every branch of origin, so the answer counts what origin holds today; that moves remote-tracking refs and nothing else.

4.12 branch delete

```
$ pixi run cgitsync branch delete <branch> [--private] [--gts <snapshot.gts>]
```

Deletes a closed branch, tree-wide. A branch that is not closed is refused: close it first. The ledger is read first: a move a close already recorded, and that still resolves, is reused and never recorded twice, so a branch closed by **branch close** records nothing new here. Anything not yet kept — a branch closed before **ancestors** existed — is kept and recorded exactly as a close does. Then the ledger is verified, and only then is the branch deleted, on origin first and then locally, leaf-first; a local branch is removed only while it still points at the commit that was checked. A step that fails refuses with nothing deleted.

4.13 add

```
$ pixi run cgitsync add [PATH ...] [--gts <snapshot.gts>] [--dry-run]
```

With no `PATH` arguments (the default), stages all changes across the tree (`git add -all`), leaf-first. Requires **READY**; tree stays **READY**. With `-dry-run`, **cgitsync** prints the leaf-first execution plan and does not mutate repositories.

With one or more `PATH` arguments, each is resolved (relative to `CGSHOME`, or absolute) to the one repo in the tree that owns it and staged there individually (`git add - <path>`), leaving every other repo untouched. A path outside every repo in the tree raises an error immediately, before anything is staged.

4.14 rm

```
$ pixi run cgitssync rm PATH [PATH ...] [--gts <snapshot.gts>] [--dry-run]
```

Removes one or more tracked files, each resolved the same way as `add`'s targeted form: to the one repo in the tree that owns it, where it is deleted from disk and the removal staged (`git rm - <path>`). A plain tracked file only — a path that resolves to a directory, or that does not exist, raises an error rather than partially applying, and a path outside every repo in the tree raises before anything is removed. Requires `READY`; tree stays `READY`. With `-dry-run`, `cgitssync` prints the plan without mutating repositories.

Distinct from and unrelated to `submodules import`' internal use of `git rm -cached` (index-only, dropping a gitlink while keeping the child's working tree) — `rm` deletes the file from disk.

4.15 commit

```
$ pixi run cgitssync commit <message> [--gts <snapshot.gts>] [--no-stage] [--dry-run]
```

Commits changes across the tree, leaf-first. Loads the `READY` registry from `-gts`. Repositories with no staged changes after the optional `git add -all` step (suppressed by `-no-stage`) are silently skipped. Requires a `READY` tree; the tree remains `READY` on success. Commit preflight now emits warnings for actionable workspace issues such as dirty worktrees, ahead branches, or stale recorded `commit_sha` values. With `-dry-run`, `cgitssync` prints the planned stage/commit sequence without performing git writes.

The commit message conventionally ends with `CGS#VERSION`.

4.16 push

```
$ pixi run cgitssync push [--gts <snapshot.gts>] [--dry-run] [--force-protocol {ssh,https}]
```

Pushes all repositories to their configured remotes, leaf-first. Loads the `READY` registry from `-gts`. Each repo is pushed to `entry.remote_name` (defaulting to `"origin"`) on its currently resolved branch. Refreshes the stored hash in `GitTree`. Requires a `READY` tree; the tree remains `READY` on success. Push preflight blocks detached HEADs, missing remotes, unresolved merges, and behind/diverged branches; it warns when the local branch is ahead or when the loaded snapshot records an older `commit_sha`. With `-dry-run`, `cgitssync` prints the push plan without mutating any repository. `-force-protocol` behaves exactly as described under `pull` above.

Folds this project's own memory first, when one is mounted. Before pushing anything else, `push` folds whatever has accumulated in `.cgitssync` into `.cgitssync/.memory` and sends it — the same work `cgitssync memory push` does by hand, run automatically the moment a command was already about to reach a remote. This happens regardless of `-private`: a memory, once adopted, settles exactly as much as an ordinary push. Nothing happens when the tree declares no memory at all. A memory that is declared but not yet adopted, or one that cannot reach its own remote for any reason, prints a warning and does not block the push — the memory's own trouble is never a reason to fail somebody else's publish. `-dry-run` lists `cgitssync memory push` first in its plan under the same condition the real run would perform it.

4.17 freeze-release

```
$ pixi run cgitssync freeze-release <name> <message> [--gts <snapshot.gts>] [--dry-run]
    [--force-protocol {ssh,https}]
```

Minimalist release workflow from a READY tree. It chains public operations in order: `add` `-> commit` `-> pull` `-> push`, then it creates and pushes the shared tag and writes the `.gts` snapshot. It is the one freeze procedure: there is no bare `freeze` command and no force variant. To release from a branch that has fallen behind, run `pull -force` first (it refuses while a commit exists that no remote holds, so commit and push before it), then `freeze-release`. The release name becomes the shared tag and snapshot label; the message is used for the release commit.

The `pull` step is skipped (not attempted) when the current branch has no upstream configured yet — e.g. a branch just created and checked out this same session, never pushed before. There is nothing to pull in that case; `push` (already upstream-aware, adding `-u` automatically the same way plain `git push -u` would) publishes the branch on its own. On any branch that already has an upstream, the workflow is unchanged.

`-force-protocol` is forwarded to the `pull` and `push` steps above, behaving exactly as described under `pull`; the remote rewrite it makes persists, so the final release’s own tag push picks it up too without needing the flag itself.

The release also records a `release` row on the ledger entry it writes: the installed package’s own SemVer and build counter, plus `<name>` as the tag actually applied. Name the release `v<semver>` (for example `v3.1.0`) to keep the tag and the recorded SemVer in agreement.

The freeze step stages and commits, creates and pushes the shared release tag, refreshes SHA/state, and writes a `.gts` snapshot, leaf-first. Its preflight checks remote availability, branch alignment, detached HEADs, unresolved merges, and tag absence. Dirty worktrees and stale recorded `commit_sha` values are reported as warnings because the freeze will stage, commit, and snapshot them. The snapshot also includes a deterministic `freeze_manifest` section that records freeze invariants (immutable snapshot, validated workspace, synchronized tag, `release-name`, ledger checkpoint) and declares `launch_state` as the canonical restore operation. Its immutable snapshot filename includes the release, for example `gts-000001-release-2026.05.gts`, while the ledger id remains `gts-000001`. To return to a release later, run `checkout <name> -ref-kind tag`. With `-dry-run`, `cgitssync` prints the planned add/commit/tag/push graph for the loaded workspace without mutating it.

Folds this project’s own memory first, same as push. `freeze-release` (and `tag`, which folds the same way on its own combined commit-tag-push) crosses the `.cgitssync` $\rightarrow$ `.cgitssync/.memory` frontier before doing anything else, under the same conditions `push` does. A release folds twice in one run — once via its `push` step, once via the freeze step — which is harmless: a fold with nothing new pending commits nothing and only re-sends the branch.

4.18 submodules report and submodules import

```
$ pixi run cgitssync submodules report REPO_ROOT [--recursive]
$ pixi run cgitssync submodules import REPO_ROOT [--recursive]
```

Reads `REPO_ROOT/.gitmodules`; `report` lists, and `import` converts, every declared git submodule to a plain ComplexGitSync nested clone. This is the whole job: turning gitlinks into plain clones on disk. It does not author a `.cgs` — `.gitmodules` never records the root’s own identity, and a checkout worth importing already has a `.cgs` (hand-authored) or can get one from `discover` (§1 “discover”), before or after importing.

`submodules report` is the *dry run*: it prints each submodule’s name, path, URL, and branch, without touching the repository.

`-recursive` (same name and meaning as `git submodule update`'s own flag) also converts any submodule that itself has its own checked-out `.gitmodules`, at any depth — root first, not leaf first: converting a submodule stages changes in its own working tree, and converting a deeper level first would make the shallower level's own preflight check (is the submodule's working tree clean?) see that staged dirt and reject its own conversion; converting the shallower level first avoids this, since `git rm -cached <path>` only ever touches the parent's own index, never the submodule's working tree. A submodule path that was never checked out (`git submodule update -init` not run for it) has no `.git` to read a nested `.gitmodules` from and is invisible either way — the same ceiling `discover` already has for itself. Without `-recursive`, behavior is unchanged: exactly `REPO_ROOT/.gitmodules`, one level.

The report lists every level together. Each entry's `path` is counted from `REPO_ROOT`, and `declared in` names the `.gitmodules` file it was read from. A submodule's own `.gitmodules` path means nothing on its own once more than one repository is involved — in `cawaqsviz`, two of the three submodules are called `docs/...` by their own repository.

`submodules import`:

- Verifies every submodule's working tree is clean (`git status -porcelain` must be empty apart from that repository's own `.gitignore`); rejects dirty trees immediately. The `.gitignore` exemption exists because `cgitsync` writes that one file itself in every repository holding a child, so it is routinely dirty in exactly the tree being converted; no conversion can harm it, since `git rm -cached` never touches the child's working tree.
- Runs `git rm -cached <path>` per submodule, dropping the gitlink from the index while leaving the child's working tree and `.git` directory intact (no re-clone, no local history lost).
- Removes the converted stanza from `.gitmodules` (deletes the file entirely when all stanzas are converted) and stages the change.
- Appends `<path>` to the parent's `.gitignore` using the same helper that the `.gitignore` lifecycle sync uses.

After importing, review and commit the staged changes manually:

```
$ git -C REPO_ROOT status          # deleted .gitmodules, modified .
    gitignore
$ git -C REPO_ROOT commit -m "chore: retire git submodules"
```

Note: `ComplexGitSync` stores no credentials and has no private-repo authentication story. If a submodule's upstream is private and the ambient environment lacks access, the subsequent `cgitsync initialise` or `pull` will fail at the clone step. The `submodules import` step itself (which only touches the local index and working tree) will still succeed.

4.19 submodules init

```
$ pixi run cgitsync submodules init REPO_ROOT [--cgs FILE]
    [--max-depth N] [--dry-run] [--force] [--force-protocol {ssh,https
    }]
```

Adopts a submodule-based checkout in one command: `discover` the repositories under `REPO_ROOT`, write the drafted `.cgs`, `initialise` the tree in place, then convert every submodule with `submodules import -recursive`. `REPO_ROOT` is a project cloned and `git submodule update -init -recursive`'d by hand; the command performs no clone of the root itself.

The conversion must come last, and this command is the reason why. `initialise` adopts the root in place but deletes and re-clones every *other* repository straight from its remote,

and those remotes still declare submodules — so a conversion run first is undone for every repository except the root. Nor can a second pass repair that: `submodules import -recursive` follows the submodule graph declared by the root’s own `.gitmodules`, which the first pass removed, so no deeper level stays reachable. Running the two commands in the other order by hand produces a half-converted tree that looks finished.

Three conditions are checked before anything is written, so a run that cannot work fails while the checkout is still untouched:

- the discovery resolved at least one repository (skipped when `-cgs` supplies a file, which is then the authority);
- the project name matches the directory name — `CGSHOME` is resolved as `<parent>/<project-name>`, so a mismatch would send `initialise` to a sibling directory that does not exist;
- `REPO_ROOT` has a `.gitmodules`. Without one there is nothing to convert, while the clone step would still delete and re-clone every non-root repository, discarding any uncommitted work in them. `-force` overrides this one.

`-dry-run` prints the discovery and the submodules that would be converted without writing a `.cgs`, cloning, or touching any repository. `-cgs FILE` uses an existing `.cgs` instead of writing one (or writes the draft there, when `FILE` does not exist). `-max-depth N` bounds the discovery step, which is otherwise unbounded — most trees never need it. `-force-protocol` applies to the clone step only; discovery reads each repository’s configured remote as it is.

The command stops at a `READY` tree with the conversion staged but not committed — it touches every repository that held a submodule, and some of those may belong to other people — and prints the `branch create/checkout/add/commit` sequence to run next.

4.20 memory

```
$ pixi run cgitssync memory status [--search-dir DIR]
$ pixi run cgitssync memory list  [--branch NAME] [--search-dir DIR]
$ pixi run cgitssync memory show <state> [--full] [--search-dir DIR]
$ pixi run cgitssync memory explore [--branch NAME] [--timeline] [--
  search-dir DIR]
```

Looks at what this workspace remembers. A **State** is one recorded snapshot of the tree, named by its own content; the **ledger** holds one entry per operation, saying when that State was seen and by which tool versions.

A workspace whose `.cgs` declares no memory — every install from `install.cgs` — gets one made for it, locally, the first time a command records something: a repository at `.cgitsync/.memory` with no remote. `memory push` folds and commits into it and never sends it anywhere, even if a remote is added by hand; `memory status` says it is local and prints the `.cgs` line that publishes it. Declaring that entry and running `memory adopt` is the opt-in; publishing a memory is a developer’s privilege.

A user’s tree or a developer’s. `status` prints `profile=user` or `profile=dev` on its summary line, and `status -json` carries the same word as `profile`. A tree whose `.cgs` holds no private repository is a user’s, and its memory never leaves the disk. A tree holding at least one private repository, read-only or not, is a developer’s, and its memory is meant to be sent to the repository its `.cgs` declares. When a developer’s `.cgs` declares none, the work is still recorded locally, and the first command that records something asks, in a terminal only, for the provider (default `github`), the owner (the owner of most of the tree’s private repositories, else the project’s own) and the name (default `.memory`); it shows the command that creates

the repository and, on a yes, runs **memory setup**. A no is asked once. Without a terminal — **-json**, CI, a Python caller — after a no, or after a step failed, it only warns that the work has no memory back-up and no global ledger record, and names the command that fixes it; **memory status** says the same.

```
$ pixi run cgitssync memory setup [--provider P] [--owner O] [--name N]
  [--cgs FILE]
```

memory setup runs **repo create**, **memory mount** and **memory adopt** (below) as one command, in that order, and stops at the first that fails, saying which and what was left done. In a terminal it asks the three questions first, with the defaults filled in; elsewhere it uses the flags and defaults as given.

- **status** — how many States, how long the chain is, when it was last written, which of **verify check**'s answers this memory is in, and the toolchain the records carry. The first entry's versions are shown beside the latest when they differ, so a chain spanning an upgrade says where the upgrade fell.
- **list** — every State, newest recording first, with the commands that recorded it. A State with no timestamp is one no entry records: history from before the ledger, or a file that arrived some other way.
- **show <state>** — the environment that State's own commits ran under, as a bare reference; that State's own topology right after, drawn exactly the way **view-tree** draws the live one (**format_view_tree**) but as it was at that State, not as it is now; then every entry naming it and what each **commit** wrote: the message, the repository, and whether the commit has been published. The argument is the State's content hash or any unambiguous prefix of it; nobody retypes sixty-four characters. Pasted back verbatim from wherever it was printed also works: the bare prefix, the **state=<hash>** label **explore -timeline** prints it under, or the full **state(<hash>)** form a ledger entry uses. **-full** prints whole messages instead of first lines, and the remote and ref each publication went to.
- **show env=<ref>** — one Environment record, in full: machine, tools, credentials and manifest digests, drawn as the same kind of tree **view-tree** draws a repo tree with. This is the detail a State's own bare **environment=env(<hash>)** reference never shows, since a State answers "what was this tree", not "what ran it". Takes the full id, a bare prefix, or the **env=<hash>** form its own argument is spelled with.
- **explore** — a memory a person can read without a hash. The default view is one row per commit this memory recorded as published, newest push first, on the branch checked out here — what a colleague pulling this branch would see. **-timeline** reads the ledger straight through instead: one row per entry, in the order it happened, so **checkout/merge/push** appear beside **commit** rather than being dropped — each row also prints the State it recorded, so a prefix to hand **show** is never more than a scroll away. **-branch NAME** asks for a different chapter of the memory. One not checked out here is read from Git, as a timeline of its entries: from its branch, its closed name, or the copy **ancestors** keeps once the branch is deleted, and the answer says which. One this clone holds nowhere names **memory clone -branch NAME** rather than answering for the wrong branch. **list** and **as-of** take the same option, and **show** looks in every chapter for a State that is not on disk; it also names the repositories whose commit, in that State, only **ancestors** still reaches.
- **as-of <time>** — what was this tree at that time? The State the ledger recorded at or before it, not the nearest and not the latest, with the **show** command to type next. A time is read as UTC unless it carries an offset (2026-09-30T17:00+02:00); a bare date means

the end of that day. A time before the first entry answers “nothing recorded yet”. When the chain does not verify, or its clock moved backwards (`verify check`’s `time-inconsistent`), the answer is printed with `answer_reliable=false` and a warning, never as if the chain were clean.

A commit marked `unpushed` exists on this machine and nowhere else. That distinction is the reason the messages are kept at all: while the repository is present `git log` is the better answer, and this record is for when it is not — a deleted branch, an archived project, a fork that went away. The ledger entry that recorded a set of messages carries their fingerprint, so editing one afterwards is a finding `verify check` reports.

`status`, `list`, `show` and `explore` are read-only; nothing they do sends a memory anywhere.

Keeping a memory when the disk does not. A memory can be a repository of its own, mounted at `.cgitsync/.memory` exactly as `.localSpec` is mounted: one repository holding every project’s memory, on a branch per project. `.cgitsync` itself stays the workspace’s own live state — States, the ledger, logs — so the mount one level inside it is checked out and merged like any other private repository, never blocked by activity that is not its own. A developer gets there with one command, run at any time: `memory setup` (§4.20). The five commands below are the same path step by step, run once per project; `tutorials/05_memory.md` walks both.

```
$ pixi run cgitsync repo create github:you/.memory
$ pixi run cgitsync memory mount [--cgs FILE]
$ pixi run cgitsync memory adopt
$ pixi run cgitsync memory push
$ pixi run cgitsync memory branch --project-branch main
```

- `repo create` — creates the repository by running the provider’s own tool (`gh`, `glab`, `tea`), which you have already signed in to. ComplexGitSync stores no credential and sends none. A repository that already exists is an ordinary success; a tool that is missing or signed out prints the command to run and exits 2.
- `memory mount` — adds one entry to a `.cgs` that already exists, leaving the rest of the file, comments included, exactly as it was. Adding it twice changes nothing.
- `memory adopt` — creates `.cgitsync/.memory` fresh and turns it into that repository, on this project branch’s memory branch. It starts empty: the first `memory push` is what folds in whatever `.cgitsync` already held. When the workspace already has the local memory ComplexGitSync made for it, that repository and every commit in it are kept: adopting adds the remote and, if the repository already holds this branch’s base, joins it with one merge commit. If the two cannot be merged cleanly, nothing is adopted and the local memory is left exactly as it was; `-reboot` or `cgitsync autofix` are the ways on. `-reboot` adopts the same repository identity but skips carrying forward whatever the fallback branch already holds, so the mount’s content starts fresh instead (§4.20).
- `memory branch` — creates the memory branch another project branch will need, and pushes it. Needed once, for a memory born on a feature branch: merging `memory-dev` into `main` asks for a branch that has never existed. A merge that finds one missing names this command rather than creating the branch itself, because a branch that is not there is as likely to be a typing mistake as a new branch.

Migrating a memory mounted before `memory migrate` existed. A memory adopted before this command existed sits directly at `.cgitsync`, sharing it with the workspace’s own live state. `cgitsync memory migrate [-cgs FILE]` moves it onto the layout above — `.git`

and every tracked file travel down into `.cgitsync/.memory`, history unchanged; whatever was never tracked stays exactly where it already was, which is where the new layout wants it anyway — and updates the `.cgs` entry that declares the mount to match. A fresh `memory adopt` never needs this: it mounts at the new layout directly.

```
$ pixi run cgitsync memory init      # the .cgs entry to add, and the
    branch
$ pixi run cgitsync memory clone     # bring a memory onto a new machine
$ pixi run cgitsync memory push      # commit what it gained, and push it
```

`init` proposes and stops: `ComplexGitSync` speaks Git and nothing else, so it prints the command that creates the repository and waits. Nothing is ever pushed automatically — a machine with no network keeps a complete, verifiable memory and sends it later.

Cloning a memory refuses to write over a memory that is already here, because a memory nobody has pushed is the only copy of itself.

Starting a memory's history over. A project's shape changes — repositories added, removed, restructured — and a memory built for the old shape stops being a clean answer to "what does this project look like." `cgitsync memory reboot` closes the current chapter and opens an empty one, without losing the old one:

```
$ pixi run cgitsync memory reboot
```

In order: whatever `.cgitsync` is holding pending is folded in and pushed under the branch's current name, exactly as `memory push` already does; the tree's current shape is exported — from the loaded `.gts`, never a hand-authored file — to a permanent, versioned `.cgitsync/.memory/.cgs/<project>` (N incrementing once per reboot, never overwritten, never reused); the branch is archived, renamed both locally and on origin to `<branch>.archived-<date>`, the old name removed from origin only *after* the new one already holds the same commits, so nothing the memory ever recorded is unreachable for even a moment; and a fresh branch takes the original name, so nothing about how the memory is mounted changes. Its tracked content is cleared except `.cgs/` — the permanent, versioned record above, which a reboot must not erase — and one fresh State is written, committed, and *pushed* immediately, so the branch is live, with a real `HEAD cgitsync status` can read, and is never missing from origin: the old name was already gone from there by the time the fresh branch is built, so a bootstrap on another machine that ran in between would otherwise have found no branch under this project's name at all, and fallen back to one that never held its `.cgs`. Nothing is ever force-pushed or deleted. `cgitsync memory adopt -reboot` is the same fresh start for a mount being adopted for the very first time. Appending — the ordinary `memory adopt`, and the only thing `memory push` ever does — stays the default either way.

What a memory carries off your machine. One path: the tree's own root, with `$HOME` substituted. Every other path it records — each repository's, and any path in a command line it logged — is written against the tree as `$CGSTREE/...`, so neither your directory layout nor your user name travels with it.

4.21 self-history

```
$ pixi run cgitsync self-history add \
    --ticket NAME --goal TEXT --action TEXT \
    --worker-role ROLE --worker-vendor NAME --worker-model NAME \
    --orchestrator-role ROLE --orchestrator-vendor NAME --orchestrator-
        model NAME \
```

```

--spec-respect-score N --spec-respect-basis measured|asserted --spec-
  -respect-reasoning TEXT \
--gating-score N --gating-basis measured|asserted --gating-reasoning
  TEXT \
--quality-score N --quality-basis measured|asserted --quality-
  reasoning TEXT \
[--state-before STATE] [--state-after STATE] \
[--lint-passed|--lint-failed] [--tests-passed|--tests-failed] \
[--pushed] [--pushed-reason TEXT] [--search-dir DIR]

```

Records one piece of agent work: which ticket it served, the goal and the action taken in plain English (at most three lines each), the two agents involved — a **worker** that implemented the ticket and an independent **orchestrator** that quoted the work, each with a role from `.localSpec/AGENT.md`'s roster, a vendor and a model version — and a three-part conformity score (spec respect, `.PUBLIC/` `.PRIVATE` gating, quality of production), each part marked **measured** or **asserted** rather than blending the two.

The score is out of **100**: spec respect is scored from 0 to 33, gating from 0 to 33 and quality from 0 to 34, and the total is their plain sum. The orchestrator starts each part at its maximum and takes off points for each shortfall it can name, saying which in that part's reasoning; that method is guidance, and the tool checks only that a score is within its range (a score outside it is refused with a clear message) and that the total is the sum. Every score is shown with its maximum, for example `spec_respect=26/33 (asserted) gating=32/33 (measured) quality=26/34 (asserted) total=84/100`. The optional `-conformity-explanation` is a short account, in the orchestrator's own words, of how the total came about; it is stored with the record and printed under the score.

Two facts the command fills in itself, because this project can check them directly rather than trust an agent to type them: which signed agent-contract record was in force (by its own hash, from `.agent/.distant/dev-sync/agent-contracts/current`), and this workspace's own `errors=` count from the same view `status` prints. Everything else is the orchestrator's own account. A missing signed contract is not an error — the field is simply empty.

`-state-before/-state-after` are **verified against the ledger**, not merely shape-checked: each, if given, must name a State some ledger entry actually recorded, still on disk, still hashing to its own name — the same three questions `verify check` itself asks. A citation that fails any of them is refused, not recorded as though it were fact. `-state-after` defaults to the ledger's own most recent entry when not given.

The record is written to `.cgitsync/.self-history/`, content-addressed the same way a State or an Environment record is — the pending half. Once self-history has been adopted (below), `cgitsync memory push` folds it into `.cgitsync/.memory/.self-history/` and pushes it, *before* folding and pushing `.memory` itself, so the parent's own push always records a leaf that already exists.

```

$ pixi run cgitsync self-history adopt [--owner NAME] [--branch NAME]
  [--search-dir DIR]

```

Self-history rides `.memory`'s own lifecycle rather than needing a lifecycle of its own. `self-history adopt` is the *one* place this is ever decided: it writes `config-memory.cgs` once, for a brand-new project or as the explicit retrofit for a `.memory` adopted *before* self-history existed. From then on, `cgitsync memory adopt` follows that decision automatically, silently, whenever a project's own memory branch inherits it from a fallback base branch that already carries it — the same mechanism that already lets a new project's memory share history with another. A project that has never adopted self-history has no such file anywhere, so this sees no new behaviour at all. Unlike that automatic follow, asking for `self-history adopt` explicitly raises when the remote is not there, rather than quietly doing nothing — asking for it means wanting to know why it did not work.

`cgitsync memory clone` brings self-history back too, the moment it was adopted upstream; `cgitsync memory reboot` leaves it entirely alone, on purpose — a reboot starts `.memory`’s own history over, and the record of what happened, including across the reboot, is exactly the thing that should survive it.

```
$ pixi run cgitsync self-history list [--search-dir DIR]
```

Every self-history record this workspace holds, folded and pending, oldest first — consultation only. A workspace that has never adopted self-history, or has adopted it but written nothing yet, prints that plainly rather than erroring.

4.22 verify check and verify repair

```
$ pixi run cgitsync verify check [--search-dir DIR] [--json]
$ pixi run cgitsync verify repair [--search-dir DIR] [--json]
```

`verify check` says which of five things is true of this workspace’s recorded history. One of them, never a blur of two:

Answer	Exit	Means
<code>verified</code>	0	A chain was read and every link held.
<code>no-history</code>	0	Nothing has been recorded here yet. Not a failure — a new workspace is not a broken one.
<code>legacy</code>	1	History exists, in the single-file <code>.lgr</code> register, which carries no chain. Readable, not verifiable: an edit to it leaves no trace.
<code>corrupt</code>	1	A chain was read and it does not hold.
<code>time-inconsistent</code>	1	The chain held, but its own timestamps move backwards somewhere. Your history is intact; the clock that stamped it was not.

`legacy` exits non-zero on purpose. The question is “is this history intact?”, and “I cannot tell” is not a yes: a build that gates on `verify check` must not pass because the evidence is in a format nobody can check. Every workspace created before the hash-chained ledger is written is in exactly that state.

`time-inconsistent` is deliberately not `corrupt`. Every link checked out — nothing was rewritten — so the two answers send you to look at different things. A clock corrected mid-session, a restored virtual machine, or a machine that disagreed about the hour all read like this, and so does an entry somebody backdated: a date cannot be moved backwards without contradicting the chain around it. It still exits non-zero, because something is wrong even though your history is not.

When there *is* a chain, what is checked: every entry’s `prev` must match the previous entry’s `entry_hash` (`BROKEN_LINK`), every entry’s own `entry_hash` must match its recomputed canonical hash (`BAD_ENTRY_HASH`), `seq` numbers must be gap-free and unique (`SEQ_GAP/SEQ_DUPLICATE`), and the cached `HEAD` must agree with the recomputed head (`HEAD_STALE`).

`CGSHOME` is located the same way as `status`: from `-search-dir`, else `$CGSHOME`, else the current working directory’s ancestors.

`verify repair` runs the same check, then corrects a stale `HEAD` cache to match the recomputed true head. **Entries themselves are never rewritten or deleted** by either subcommand — a broken chain is reported, not silently healed, since a ledger that can be edited back to looking clean provides no evidence of anything.

The States on disk are checked too. Beyond the chain itself, `verify check` cross-references what the entries say against what is there: an entry naming a State that is missing (`MISSING_STATE`), a stored snapshot whose content no longer hashes to the name it is filed under (`STATE_DIGEST_MISMATCH`), and a State on disk that no entry records (`ORPHAN_STATE`). The second is what naming a State by its content bought: a stored snapshot can no longer be edited quietly.

Where the history lives. `.cgitsync/lgr/`, one file per entry, appended to by every command that writes a State. Each entry records the versions that produced it — `cgitsync`, `git`, `pixi`, and the data backends `dvc` and `git-lfs` — inside the entry hash, so an edited version string is as detectable as an edited command. The older single-file `.lgr` register is still read, and is no longer written.

4.23 `env show` and `env check`

```
$ pixi run cgitsync env show [--search-dir DIR]
$ pixi run cgitsync env check [--search-dir DIR] [--cgs FILE]
```

`env show` observes the environment that can make the current tree usable. It reports the machine architecture, Pixi platform, operating system, kernel and libc; raw and parsed versions for Python and the tools `ComplexGitSync` drives; provider authentication as a fact; and the paths and digests of recognised manifests in the tree. A missing tool is printed as `none`, never as an empty value. Tokens, keys, user names, key paths, and paths outside `$CGSTREE` are never part of an Environment record.

`env check` compares that observation with requirements from the loaded tree's source `.cgs`. Pass `-cgs FILE` to check an explicit authoring document instead. Missing tools and versions older than the declared minimum are drift: the command names each mismatch and exits non-zero, which makes it suitable for continuous integration. Extra observed tools are reported as information and do not make the check fail.

The check is separate from tree operations on purpose. `initialise` and `pull` may warn about drift, but a mismatch never blocks them. Both environment commands are read-only and install nothing.

The optional authoring declaration names the repository where the environment is installed and requirements that cannot be inferred from a lock file:

```
environment_root = "my-project"

[environment]
tools = { git = "2.43", pixi = "0.66" }
compilers = ["cc"]
system_libraries = ["libssl"]
services = ["postgresql"]
manifests = ["Cargo.lock"]
```

The root must name a configured repository (or `root`). Manifest contents remain in Git; the Environment record stores only tree-relative paths and digests.

4.24 `view-tree`

```
$ pixi run cgitsync view-tree [spec.cgs|snapshot.gts] [--depth N] [--collapse REPO]
```

Renders a deterministic terminal tree that focuses on repository topology and operational state. Each line reads:

name (node_type) [sync_state] @sha br=<branch> [fb=<fallback>]

br= is printed for *every* repository, including the common case where the branch is simply the tree's default, so that reading **view-tree** tells you the branch each repository targets without opening the **.cgs**. **fb=** appears only when the declared fallback branch differs from that target, since a fallback equal to the target says nothing. Use **-depth** to limit visible levels and repeat **-collapse** to hide selected subtrees.

On a tree with private mounts, **br=** is where you see pinning at work: the unprivate repositories carry the branch you moved the tree to, and each private mount still shows its own.

Branch Topology Propagation Rules (T35):

1. **Reference branch:** The root repository's current branch is the canonical reference for all repos.
2. **Leaf-to-root inheritance:** Branch targeting flows root-first via **propagate_global_branch**, verified on-disk by **ComplexGitSyncClient.validate_topology()** (Python API only) without issuing any git writes.
3. **Allowed divergence:** Repositories in a frozen (**TAG**) state produce a **tag_divergence** diagnostic that does *not* make the topology incoherent.
4. **Incoherent states** (blocking): **misaligned_branch** (different branch from root) and **detached_head** (detached HEAD without a tag reference).

4.25 What cgitsync promises a script

Exit codes. Three, meaning the same thing in every command:

Code	Means
0	The command did what was asked.
1	It ran, and the answer is no — a merge conflict, a tree that is not READY , a verification that found something.
2	It could not run — bad arguments, no workspace, a missing or unreadable file.

The distinction that matters to a script is between “I asked and the answer is no” and “I could not ask”. **validate** is the one command that judges a document rather than acting on one, so an invalid document is its answer (1); every other command exits 2 on the same document because it could not run at all.

An expected failure prints one line on **stderr** and no traceback. A traceback is therefore a defect report, not a usage error — which is the point of mapping the expected ones.

-json. **status**, **verify check** and **verify repair** accept it. Each prints one JSON object on **stdout** and nothing else; every human-facing line goes to **stderr** instead, and the exit code is unchanged, so a caller may use either signal. A failure prints one object too, carrying **"status": "error"** and the exit code, rather than leaving a pipe with nothing to parse. **-help**, **-version** and a command line that fails to parse are deliberately not JSON: the flag itself may be what failed.

The object carries **schema_version**. The promise attached to it is **additive only**: new fields may appear, existing fields do not change meaning and do not vanish.

What is stable, and what is not. This is the contract the version number is measured against: a change that breaks a line marked stable is a new major version.

Surface	Promise
Command names and documented flags	Stable within a major version.
Exit codes	Stable within a major version.
-json output	Additive only , as above.
.cgs and .gts grammar	Versioned in the file, and the version is read on load.
Python modules under src/ComplexGitSync/	Not a public interface. ComplexGitSyncClient is the CLI's own implementation; import it and a refactor may break you, with no deprecation owed. The CLI is the product.
verify	Experimental. The ledger it reads is still changing, so its output and findings may change.

4.26 Where a command works: the default workspace

Every command that is not given an explicit path finds its workspace from three inputs, in this order: `-search-dir`, then `$CGSHOME`, then the ancestors of the current directory. When none of them lands on a `.cgitsync` directory there is a fourth answer: an empty workspace of ComplexGitSync's own, under `$HOME/.cgs` (or `$CGSPATH` when that is set).

That workspace is a real directory holding a valid `.gts` with no repositories in it. An empty tree is not an error — it is a project that has not started — so `status` answers `no living project yet`, names the three commands that start a project, and exits 0.

- **Created once, then reused.** Its path is recorded in `$HOME/.cgs/default`; later runs read that pointer instead of minting a second empty workspace.
- **It never chooses for you.** When other workspaces exist under the root they are listed, each with the `export CGSHOME=...` line that would select it, and none is selected automatically.
- **It never overrides `-search-dir`.** A directory you named that holds no workspace is an error, not an invitation to work somewhere else.

The `use_case` field. Every discovery line also says which of the two ways of running is in force: `nested` when the ComplexGitSync being executed lives inside the workspace it manages, `standalone` otherwise. It is observed and printed, never obeyed — nothing behaves differently because of it. It exists so that a user who believes they are in one case while standing in the other is told.

4.27 status

```
$ pixi run cgitsync status [--gts FILE]
```

Summarises tree readiness and per-repository sync state. The table reports the scope, local branch, upstream branch, local worktree state, ahead/behind tracking counts, current HEAD, and the commit recorded in the loaded `.gts`.

The `cgitsync_branch` field. The `summary` line carries `cgitsync_branch=<branch>`: the branch the project is on, which is the branch its root repository is on. It is what `cgitsync checkout` sets (`branch create` makes a branch without moving onto it), and what every other repository follows.

The table shows a branch per repository and they are not all the same on purpose — a `private/local` repository keeps this project's settings on a branch named after the project, so a project on `apoub` shows `ComplexGitSync_apoub` there. The field is the one place that says which of those branches is the project's own.

Shown	Means
<code><branch></code>	The branch the root repository is on.
<code>detached</code>	The root is parked on a commit rather than a branch. <code>cgitsync checkout <branch></code> puts the tree back.
<code>unknown</code>	There is no branch to report: no project has been loaded, or Git could not be asked.

The field is printed in every state. A field that disappears when the answer is awkward is harder to read, by eye or by script, than one that says it does not know.

The `SCOPE` column. It says, in three words, what kind of repository each row is and whether you may write to it:

Shown	Means	In the <code>.cgs</code>
<code>project</code>	this project's own; write freely	no <code>private</code>
<code>private/local</code>	shared, and yours to write	<code>private</code> , writable
<code>private/distant</code>	shared, read-only	<code>private</code> alone

`private` means the repository is shared with other projects — a configuration repository. `local` then means this project may write to it, normally because it sits on a branch named after your project; `distant` means you only read it. A repository nested inside a `private` one is shown the same way, because it is just as shared.

`commit` and `push` reach the `project` rows; `-private` reaches the `private/local` rows; nothing writes to a `private/distant` row. When any `private` row is present, the table prints a legend saying so.

The `SYNC` column. It says how the branch a repository is on stands against the branch it tracks — the one named in `UPSTREAM_BRANCH`, which reads `-` when there is none. There are six values:

Shown	Means
<code>synced</code>	Level with the upstream.
<code>ahead(+N)</code>	N commits here that the remote does not have. <code>push</code> sends them.
<code>behind(-N)</code>	N commits on the remote that are not here. <code>pull</code> fetches them.
<code>diverged(+N/-M)</code>	Both, from a common ancestor. <code>autofix</code> (§4.6) diagnoses which of <code>merge</code> or a chain-aware splice is safe; <code>pull -force</code> refuses to run while this workspace's own N commits exist nowhere else.
<code>no-upstream</code>	This branch was never pushed, so there is nothing to compare it to. Normal for a branch you just made, and for a <code>private/local</code> repository that only <code>push -private</code> ever sends.
<code>unknown</code>	The branch names an upstream that does not resolve. <code>pull</code> or <code>push</code> repairs it; if it persists, the remote is unreachable or the ref was deleted.

The last two are worth telling apart. **no-upstream** is a normal state of a branch nobody has pushed yet; **unknown** is a fault. They used to share the one word **unknown**, which made the column not worth reading — an ordinary new branch looked exactly like a broken one.

unmeasured in the summary line. The summary counts the **no-upstream** and **unknown** rows as **unmeasured**, separately from **ahead** and **behind**. Those rows were previously folded into **ahead=0 behind=0**, which reported a measurement nobody had taken. A repository nobody could measure is not a repository that is level, and the summary no longer says the second when it means the first.

The memory mount reading dirty mid-push, or HEAD ending in * right after, is normal. If the tree mounts its own memory at **.cgitsync/.memory** (§4.20), that row is an ordinary private/local repository the rest of the time — clean before and after every command except **memory push** itself, which folds **.cgitsync**’s own pending content into it before committing. One thing about it is still worth knowing:

- **HEAD ends in * right after memory push.** The ***** means the loaded **.gts** recorded a different commit than the one actually there now. **memory push** moves the memory’s real commit directly and does not write a new **.gts** itself, so the State most recently written still names the commit from before the push. The very next command that writes a State reads the memory’s true **HEAD** and the two agree again — it never needs a second **memory push** to catch up, only an ordinary next command.

Not worth chasing to a clean reading on purpose: a memory that never showed this for a moment would not be recording what **memory push** just did.

4.28 merge

```
$ pixi run cgitsync merge <branch> [--into TARGET] [--private] [--ff-only|--no-ff] [--dry-run] [--resolve]
```

Merges a branch across the tree, leaf-first. The argument is always the **project’s** branch; each repository works out what that means for itself. A project-owned repository merges that branch. A private/local repository merges the branch derived from it (**<base>_<branch>**), because that is where its settings for that project branch live — it has no branch of the project’s own name at all.

-into: merging into a branch you are not on. Without it, **merge** merges into whatever is checked out, so reaching another branch means running **checkout** first. **-into TARGET** names the target and does both halves in one command.

That is not a convenience when the tree being merged **contains the ComplexGitSync you are running** — the developer checkout, which installs itself editable. A separate **checkout** replaces that build, and the merge typed after it runs under the older one, against a workspace the newer one wrote. A single command finishes under the build it started with, because Python has already imported its modules.

Every repository is checked before any is touched, so a conflict or a missing target leaves the whole tree where it was: still on the source branch, nothing checked out and nothing merged. A fast-forward is reported as one, which is usually why a repository looks untouched afterwards. **-into** never creates its target: a branch that is not there is as likely to be a typing mistake as a new branch, so it is named and the command stops. It cannot be combined with **-resolve**, which promises the opposite about a conflict.

`checkout` warns when it is about to install a different build of the tool, and still performs the checkout — looking at an older branch is legitimate, being surprised by it is not.

A `-private` or `-all -into` call only ever touches part of the tree, and the rest is meant to still be somewhere else until it too is asked to move. So a second scoped call — `-private` after the plain form, or `-all` after either — finishes what the first one left behind, rather than refusing those repositories for not having moved yet.

`-private` merges the writable configuration repositories instead of the project's own. Read-only ones are never merged.

A conflict in any repository leaves no repository merged.

Every repository in scope is checked before any is merged, using a read-only test that touches neither worktree nor index. A tree-wide merge that stopped halfway would leave a workspace no `.gts` describes and no command undoes, so it is prevented rather than reported. `-dry-run` prints the plan — and, importantly, the branch each repository would actually merge, and whether it would merge at all.

The refusal names the branch and the files. When `cgitsync` refuses, it names every blocked repository and, under each, the branch it was merging and every file that conflicts:

```
merge refused; no repository was merged: ComplexGitSync: merging 'my-  
feature' conflicts in tests/unit/test_documents.py
```

A file that both branches changed is only listed when it genuinely conflicts. Binary files count: a tracked PDF changed on both sides conflicts just as a source file does, and is reported the same way. `-dry-run` shows the same list without merging.

-resolve: fix the conflict instead of reading about it. `-resolve` merges one repository at a time and stops at the first that conflicts, then opens that repository in your merge tool.

-resolve gives up the guarantee above.

Repositories merged before the conflict stay merged, so the tree can be left partly merged. That is the price of reaching a conflicted worktree at all, and it is why this is opt-in: the command prints the warning before it writes anything, then names what it merged, where it stopped, and which repositories it never reached.

Your own `merge.tool` is used whenever you have configured one. Otherwise VS Code is suggested when it is available, passed for that one call — your Git configuration is never written. With no merge tool available at all, the command prints the command to run by hand and stops, rather than failing.

Check out the target first. `merge` brings a branch *into* the one the tree is on, exactly as `git merge` does. Running `cgitsync merge multi-branch` while the tree is still on `multi-branch` would merge it into itself — which Git reports as success while doing nothing — so `cgitsync` refuses and says to check out the target branch first.

Landing a feature branch is therefore two commands:

```
$ pixi run cgitsync merge multi-branch  
$ pixi run cgitsync merge --private multi-branch
```

4.29 pull -private

```
$ pixi run cgitSync pull --private
```

The other direction. While a project feature branch is open, work lands on a private/local repository's base branch and its derived branch does not see it. This fetches and merges the base branch into each one, so a settings branch does not drift behind the project's. A repository already on its base branch has nothing to take.

5 Document Formats

.cgs means **C**omplex**G**it**S**ync specification, .gts means **G**it**T**ree**S**tate snapshot, and .lgr means **L**ocal **G**it **R**egister.

5.1 Local Authoring Spec (.cgs)

A TOML file that describes the repository tree to manage. It is the only hand-authored input; it is never modified at runtime. The normal form has two top-level keys:

```
project = "CGSi11"
repos = [
  "gitlab:CGS_test/CGSi11",
  "gitlab:CGS_test/CGSi12",
  "github:flipoyo/CGSi11",
]
```

Repository identifiers have the form `provider:owner/repository`. Nested GitLab namespaces are supported; the final path segment is the repository name. The document pipeline is:

PARSE -> NORMALIZE -> VALIDATE -> canonical CgsDocument

The canonical providers and hosts are:

Provider	Host	Remote generation
github	github.com	SSH and HTTPS
gitlab	gitlab.com	SSH and HTTPS
codeberg	codeberg.org	SSH and HTTPS
custom	explicit URL required	SSH and HTTPS

Repository-ID parsing is provider-independent and deterministic. For example:

```
codeberg:GX4G/GX4G
```

normalizes to `gitprovider = codeberg`, `project_owner_name = GX4G`, and `repo_name = GX4G`. A custom identifier must use an inline table so the host is explicit:

```
{ repository = "custom:team/tool", gitprovider_url = "https://git.
  example.com" }
```

No host is inferred for `custom`. The configured access protocol then selects `git@host:owner/repository.git` or `https://host/owner/repository.git`.

The textual grammar and its only parser, `parse_repo_id()`, live in `cgs_format.py`. The provider model in `git_repo.py` receives the already-separated provider, owner, and repository fields and composes remotes; it does not parse authoring shorthand.

The `.cgs` format pipeline is deterministic and offline. A syntactically valid repository may therefore reference a host, owner, branch, or tag that is not currently reachable. Remote existence and reference availability are checked only by explicit Git operations during runtime resolution and execution.

Normalization supplies `default_branch = main` (see “Branches in a `.cgs`” below for the full chain), makes `fallback_branch` equal to that default, selects `access_protocol = ssh`, enables `nested_config = auto`, and uses the repository name as `relative_path`. If exactly one repository name matches `project`, its path is inferred as `..`.

Advanced inline tables are used only where a value differs from those defaults:

```
project = "demo"
repos = [
  "github:owner/demo",
  { repository = "github:owner/docs", relative_path = "documentation",
    access_protocol = "https" },
]
```

The legacy advanced `[project]` and `[[repos]]` tables remain accepted for project-wide branch settings, custom provider URLs, tags, and other explicit overrides. Unknown providers, malformed identifiers, duplicates, and missing `project` or `repos` are rejected before a `GitTree` is built.

Two checked-in files document this boundary. Copy `examples/template.cgs` for normal authoring. Its equivalent `examples/normalized_template.cgs` expands every inferred field and is provided as a developer reference for the canonical `CgsDocument` shape. The two files are regression-tested for semantic equivalence.

The repository also keeps three checked-in, didactically ordered tutorials under `tutorials/` — easiest first, see `tutorials/README.md` for the guided order — exercising this authoring surface end to end: `tutorials/01_first_multi_repo_workspace.md` walks a fully exercised local sandbox topology (CGSi11) through the complete CLI lifecycle; `tutorials/02_onboarding_a_real_build_tree.md` hand-authors a `.cgs` for the larger, real-world `cawaqs` tree and the point where `cgitsync` hands off to the project’s existing make-based build workflow; and `tutorials/03_adopting_a_real_project.md` covers the real-world `cawaqsviz` project (not synthetic), which has no `.cgs` of its own and still tracks three repositories on two levels as git submodules, as one verified procedure: clone, check out the submodules, adopt the whole tree with `submodules init (discover + initialise + submodules import`, in that order), then `branch create/checkout/add/commit/push`.

`examples/cawaqsviz.cgs`, the hand-authored alternative mentioned in `tutorials/03_adopting_a_real_project.md` above, describes `https://gitlab.com/cawaqs/gviz/cawaqsviz` and exercises several patterns above at once: a nested GitLab subgroup path (`cawaqs/gviz/cawaqsviz`, three segments rather than a plain `owner/repository` pair), an explicit `relative_path = "."` on the root entry rather than relying on the project-name auto-mount rule, two GitHub children mounted at non-default relative paths (`external/HydrologicalTwinAlphaSeries` and `docs/CWV_user_guide`). Neither child has a `.cgs` of its own, but no override is needed for that: the default `auto` nested discovery resolves a repository with zero `*.cgs` matches as a normal leaf. This file previously shipped with a nonexistent repository identifier and an invalid `default_branch`; it was corrected and given real end-to-end test coverage (`tests/integration/test_cgsi_topology.py`, and one verified run against the live repositories). See `tutorials/03_adopting_a_real_project.md` for the full worked example, and `bootstrap` above for running `ComplexGitSync` against it without cloning `ComplexGitSync` inside the tree it manages.

Generated specifications use the same concise representation. The `GitTree.to_cgs()` convenience method delegates model projection to `cgs_format.py`; neither `GitTree` nor `GitRepo` formats TOML or implements the authoring grammar. Serialization omits every default that normalization can deterministically reconstruct and retains inline tables only for exceptional settings such as a non-default branch, path, protocol, tag, custom host, or discovery rule.

The supported round trip is semantic:

```
.cgs -> CgsDocument -> GitTree -> CgsDocument -> .cgs
```

Parsing and normalizing the generated file must reproduce the initial canonical document. Whitespace, comments, and TOML table layout need not be byte-for-byte identical.

5.2 Branches in a .cgs

Two different questions, both decided per repository, and easy to conflate because both are spelled “branch”: **which branch does this repository target**, and **what does cgitsync clone if the target turns out not to exist on the remote**.

Field	Where it is written	Default
<code>repos[].branch</code>	one repository entry	— (unset)
<code>repos[].tag</code>	one repository entry	— (unset)
<code>repos[].default_branch</code>	one repository entry	the project’s
<code>project.default_branch</code>	the project table	<code>main</code>
<code>repos[].fallback_branch</code>	one repository entry	that repository’s own
<code>repos[].private</code>	one repository entry	<code>false</code>
<code>repos[].writable</code>	one repository entry	<code>false</code>

The target is decided by the first four, read as one chain, a tag winning outright over any branch:

```
tag -> branch -> default_branch -> project.default_branch -> main
```

`project.default_branch` is the tree’s branch: every repository inherits it unless the repository says otherwise. `repos[].default_branch` is the branch that one repository targets. This chain governs the **root** repository exactly as it governs every other entry — a **branch**, **tag** or **default_branch** declared on the root targets that ref, the same as anywhere else in the tree. (An earlier version of `cgitsync` silently ignored these three fields on the root entry specifically, cloning `project.default_branch` regardless of what the root itself declared; `DiscoverRoundTrip` fixed this, so a `.cgs` written against that version and relying on the old silence should be checked.)

The fallback is a separate question, decided afterwards, only for repositories the chain above already resolved: `repos[].fallback_branch` is consulted only when the *target* branch turns out to be absent from the remote — `cgitsync` clones the fallback instead and records that it did so, rather than failing. A fallback is never itself a target: setting only `fallback_branch` on a repository, with no `branch`, `tag` or `default_branch` anywhere in its chain, still targets whatever `project.default_branch` says (`main` by default) and merely falls back to the declared branch if that target is missing on the remote — `discover -write` drafts both fields for exactly this reason (see `discover`, above): a tree scanned on a branch that differs from the root’s own also needs an explicit `default_branch`, not only a `fallback_branch`, or re-materialising it retargets that repository at the root’s branch instead of reproducing what was scanned.

Each link in the target chain is silent. Nothing warns you that a decision was made, so **state project.default_branch in every .cgs you write**, even when the answer is `main`. A reader of the file can then see the branch the tree lands on without knowing what `cgitsync`’s built-in default happens to be. Every `.cgs` shipped with `ComplexGitSync` does this, and a test enforces it.

5.2.1 What private means

A repository is **private** when it is **shared with other projects** and must stay on its own branch.

Branch moves stop at a private repository. Tags do not.

`cgitsync branch create <name>` and `cgitsync checkout <name>` skip a private repository and leave it on its declared `default_branch`, because moving it would drag every other project that mounts it onto your branch.

5.2.2 Read-only and writable configuration repositories

A private repository is a **configuration repository**: shared with other projects rather than owned by this one. `private = true` alone means **read-only** — `cgitsync` will not write to it. Add `writable = true` for the case where the repository lives outside your project but the part you use is yours, normally because it sits on a branch named after your project that nobody else reads:

```
{ repository = "github:you/.sharedSpec", default_branch = "main",  
  private = true },  
{ repository = "github:you/.myNotes", default_branch = "MyProject",  
  private = true, writable = true },
```

Read-only is the default because the expensive mistake is writing to something shared by accident, never the reverse. `writable = true` on an unprivate repository is rejected at validation: an unprivate repository is this project's own and is always writable.

This is a **safety rail, not a permission system**. It stops a tree-wide sweep from writing where you did not mean; it cannot stop anyone running `git` in that directory by hand, and it is no substitute for branch protection on the remote.

A private/local repository gets a branch per project branch. A private/distant repository is private to its owner: you read it, only that owner writes to it, and nothing you do moves it — so a branch move leaves it exactly where it is. A private/local repository is the opposite case: it holds settings that configure *your* project and you commit to it, so it needs somewhere to record them per project branch.

`<project name>` on main, `<project name>_<branch>` on any other branch

The base is the `**project's**` name. `main` takes no suffix, because the project's main line's settings branch is simply the project's name — which is what every existing tree already has, so nothing has to move. The separator is an underscore because hyphens already occur inside branch names.

A user never types the derived name. `cgitsync checkout <branch>` puts the project's own repositories on that branch and each private/local one on the branch this rule names, creating it when it is missing.

Without this, one settings branch serves every branch of the project, and documentation written for an unfinished feature is live on the default branch describing code that is not there.

-private everywhere. Every command that touches Git takes it: ten commands, `pull` (with or without `-force`), `fetch`, `checkout`, `branch` (on `create`, `list` and `close`), `add`, `rm`, `commit`, `merge`, `push` and `tag`. `freeze-release` does not take it: it is a whole-tree release. It narrows the command to the writable configuration repositories alone, so they can be worked on without reaching for plain `git`. Read-only configuration repositories are never written to, with or without it. Four of those commands also take `-all`, described next.

-all: both halves, one command. Running every write twice — once plainly, once with **-private** — is the right default when the two changes are different changes. When they are one change, **-all** does both halves in a single pass, under a single commit message. Four commands take it: **add**, **commit**, **push** and **merge**.

Form	Reaches
<code>cgitsync <cmd></code>	this project's own repositories
<code>cgitsync <cmd> -private</code>	the writable configuration repositories
<code>cgitsync <cmd> -all</code>	both of the above, in one pass

The plain form has not moved, so nothing an existing script runs changes meaning. **-all** and **-private** cannot be combined: **-all** already includes everything **-private** selects. No form of either writes to a read-only configuration repository — “all” means every repository **cgitsync** may write to, never every repository in the tree.

The output still names the two halves apart, so giving up the typing does not mean giving up seeing which repository was written as what. On a tree with no writable configuration repository, **-all** does the project half and reports the other as empty; that is the common case, not a fault. An explicit **-private** on the same tree still stops, because there the user asked for something that is not there.

merge -all checks every repository before merging any.

That is why **-all** is one pass and never two. Two sequential merges mean two preflights: the project half would merge, the configuration half would then refuse, and the tree would be left half-merged — exactly the state **merge**'s whole-scope check exists to prevent. **-dry-run** previews the same combined scope, and **-resolve** keeps its own warning and its deliberate partial progress.

A pin covers everything inside it. A configuration repository often holds repositories of its own, reached through its nested **.cgs**. Those are just as shared as the repository holding them, so they inherit its state: **a repository inside a read-only configuration repository is read-only, even when its own entry says nothing about it**. One inside a writable configuration repository is writable, and **-private** reaches it.

A nested entry may restrict itself further — writing **private = true** on its own line, with no **writable**, makes it read-only inside a writable parent. It cannot go the other way: **writable = true** inside a read-only configuration repository is ignored, because a repository can never be more open than the one that contains it.

5.2.3 Which repositories each command touches

Command	Reaches	Why
add , commit , push	this project's own	they write this project's history
the same, -private	writable config repos	shared, but on your own branch
branch create , checkout	this project's own	a branch must not spread
tag , freeze-release	everything writable	they create <i>and push</i> a tag
pull , status	everything	reading a shared repo is the point

Preflight follows the same scope. Each command checks only the repositories it is about to touch. A configuration repository that is behind its upstream, or sitting on its own branch, never blocks a **commit** of this project's own work — that repository is not part of the commit. Under **-private** it is in scope, and its branch is then checked against the branch it is *private* to, not against the one the project is on.

Which commands accept `-private`. Exactly these: `pull`, `fetch`, `checkout`, `branch`, `add`, `rm`, `commit`, `merge`, `push` and `tag` — and nothing else.

`-private` is exclusive, not additive: it selects *only* the writable configuration repositories, so a shared repository gets its own command and its own commit message rather than being swept into this project's. Asking for it in a tree with no writable configuration repository is an error naming the read-only ones, not a silent no-op. Every scoped command reports what it left alone:

```
scope=project skipped=5 configuration repo(s) (.claude, .localSpec with
--private)
```

`tag` and `freeze-release` stop at read-only repositories because they push. Nothing is lost: a `.gts` snapshot records every repository's exact `commit_sha`, read-only ones included, so an exact tree is rebuilt from the snapshot rather than from tags.

In practice shared config mounts are usually dot-named (`.agentSpec`, `.localSpec`), and `cgitsync discover` drafts a dot-named repository with `private = true` for that reason. The rule is *shared*, not *hidden*, though: a plainly named repository that two projects mount should be private too.

The branch a repository is private to decides who sees your commits. A mount private to a branch named after your own project is yours to push freely. A mount private to `main` is read by every project that mounts it, so a commit there is published the moment it is pushed — no branch, no pull request, no review. Check which of the two you are looking at before committing to a private mount.

5.2.4 The two meanings of “force”

`pull -force` is destructive, but it loses nothing: uncommitted and untracked work is stashed (`git stash pop` restores it), and it refuses, for the whole tree, while a commit exists that no remote holds, so it never drops a commit. `-force-protocol` is not: it only switches a remote between `ssh` and `https`. The two are unrelated despite the shared word.

5.3 Repository Placement and Nested Discovery

For every non-root entry, `relative_path` is relative to the parent repository represented by the current `.cgs`; it is not relative to the process working directory. At the top level that parent is the project root. While expanding a nested config, it is the repository containing that config. The default is the repository name. A unique repository matching the project name uses `.` because it represents the current root.

The `nested_config` option controls whether traversal continues after the repository is available:

- **auto** (default) loads the sole root-level `*.cgs` file, if one exists; finding none resolves the repository as a normal leaf; multiple matches are rejected as ambiguous.

A consequence worth knowing before you publish a repository: **a repository that other people may mount inside their tree must keep exactly one `.cgs` at its root**, because `auto` has no way to choose between two. The tree's own root is exempt — it is never scanned for its own `.cgs`, since that would recurse — so a root holding two `.cgs` files works fine until the day someone mounts it. The error names both files and how to resolve it.

- **disabled** does not inspect the repository for nested config.
- A relative path ending in `.cgs` loads that exact file inside the repository, failing if it does not exist. It may not escape the repository root.

For the integration topology, `CGSi12.cgs` contains:

```
{ repository = "github:flipoyo/CGSi12", relative_path = "../CGSi12" }
```

Because the config describes the tree below `CGSi12`, the path resolves from that directory:

```
<CGSHOME>/CGSi12/../../CGSi12 -> <CGSHOME>/CGSi12
```

It therefore references the existing sibling declared by `CGSi11.cgs`, instead of cloning another `CGSi12` under `CGSi12`. Discovery recognizes the already-registered absolute path and keeps the canonical entry — this guard runs before `auto` would ever get a chance to reopen `CGSi11.cgs` through the cross-reference, so no `nested_config` override is needed here.

5.4 Git Tree State Snapshot (.gts)

A TOML file generated automatically by bootstrapping and release operations. It must never be hand-edited. It captures the exact commit SHAs and lifecycle state of the tree at a point in time.

Top-level sections:

- `[document]` — `format_version`, `schema_version`, `generated_at`, `command_origin`, `hash_algorithm`, `snapshot_hash`.
- `[project]` — project name and `root_absolute_path`.
- `[tree_state]` — `lifecycle_state`, `is_ready`, `registry_complete`.
- `[[repo_state]]` — one entry per repo; includes `commit_sha`, `sync_state`, and ref information.

Why both version fields and hash metadata exist:

- `format_version` identifies the broad document family and keeps long-range compatibility intent explicit.
- `schema_version` identifies the exact field-level contract used by validation and canonical snapshot serialization.
- `snapshot_hash` (with `hash_algorithm = "sha256"`) provides deterministic workspace identity: identical tree state produces identical digest even if volatile metadata (`generated_at`, `command_origin`) differs.

How this is used at runtime:

- During snapshot generation, `ComplexGitSync` writes `schema_version`, `hash_algorithm`, and canonical `snapshot_hash`.
- During load/validation, `snapshot_hash` is recomputed from the canonical payload and must match.
- The project-local `.lgr` register uses this canonical hash to deduplicate equivalent snapshots.

5.5 Local Git Register and Sync Ledger (.lgr)

A TOML file maintained per project at `<project-root>/<Project_name>.lgr`. It is updated automatically whenever a `.gts` snapshot is written.

Sections:

- `[register]` — current snapshot pointer (`current_snapshot_id`, `current_snapshot_hash`, `current_snapshot_path`).
- `[[snapshots]]` — list of stable `gts-XXXXXX` identifiers, one per unique workspace state, deduplicated by SHA-256 hash.
- `[[ledger]]` — append-only list of synchronisation events forming a DAG via `parent_sync_ids`.

Ledger event schema:

```
[[ledger]]
sync_id      = "lgr-000001"
parent_sync_ids = []
operation    = "clone"
timestamp    = "2026-05-20T19:48:50.159Z"
actor        = "username"
workspace_hash = "5f7c8a2d..." % 64-character SHA-256 hex digest
gts_snapshot_id = "gts-000001"
affected_repos = ["demo", "dep-a"]
```

The `workspace_hash` field equals the canonical SHA-256 digest (`document.snapshot_hash`) of the associated `.gts` file, creating a direct link between each ledger event and the immutable workspace state it records. Events are parents-before-children in DAG order.

6 Worked Examples: From CGS11 to cawaqsviz

ComplexGitSync ships a tutorial under `tutorials/` for each level below (`tutorials/README.md` lists them in order). Run through them in order: each one adds exactly one new idea on top of the last, so together they form a guided path from the simplest possible `.cgs` to a real project with no `.cgs` of its own at all.

Levels 2 and 3 have their `.cgs` checked in under `examples/`. Level 1 does not: `CGS11.cgs` lives in the `CGS11` repository itself, which is what the transcript below clones.

Every command below is a Pixi task. `cgitsync` is not a globally installed executable — always run `pixi run cgitsync ...`, never a bare `cgitsync ...` (see the §1 “CLI Overview” note above).

Level	Topology	Entry point	New idea introduced
1	CGS11	hand-authored <code>.cgs</code>	minimal spec, name-matching auto-mount
2	cawaqs	hand-authored <code>.cgs</code>	the same minimal-field style as Level 1, at 19-repo scale
3a	cawaqsviz	hand-authored <code>.cgs</code>	explicit overrides on a real, irregular tree
3b	cawaqsviz	discover	drafting a <code>.cgs</code> from a check-out, no authoring at all
3c	cawaqsviz	submodules import	migrating an existing git-submodules project

Levels 3a–3c are not three different projects: they are three different *starting points* for the same real project, `cawaqsviz`, chosen so this section can demonstrate every route ComplexGitSync

offers for arriving at a working `.cgs` — write it by hand, derive it from a checkout, or migrate it from an existing submodules setup — without inventing a fourth toy topology to do it. It is the most advanced level not because `cawaqs` is a larger tree than `cawaqs` (it is much smaller, three repos against nineteen) but because, unlike either earlier level, it has no `.cgs` anywhere upstream to copy from.

6.1 Level 1 — CGSil1: the minimal synthetic topology

CGSil1 is the easiest way into ComplexGitSync: a 4-repository, mixed-provider tree with every field left at its default. Its full `.cgs` and the rationale for each line are in §1 “Local Authoring Spec” above; `tutorials/01_first_multi_repo_workspace.md` installs it and `tutorials/02_working_with_a` works on it, with the expected output of each step. What follows is the command sequence itself.

```
# 1. Fetch the root repo and cgitSync itself (nested mode: cgitSync is
#     cloned inside the tree it will manage).
$ git clone https://gitlab.com/CGS_test/CGSil1
$ cd CGSil1 && git clone https://github.com/flipo/ComplexGitSync
$ cd ComplexGitSync

# 2. Check the spec offline, no network beyond parsing.
$ pixi run cgitSync validate ../CGSil1.cgs

# 3. Preview the topology before touching disk.
$ pixi run cgitSync view-tree ../CGSil1.cgs

# 4. Clone every declared repo and reach READY.
$ pixi run cgitSync initialise ../CGSil1.cgs

# 5. The ordinary git cycle, run tree-wide from here on --- no path
#     argument needed, the .gts snapshot is discovered automatically.
$ pixi run cgitSync pull
$ pixi run cgitSync add
$ pixi run cgitSync commit "my commit message"
$ pixi run cgitSync push

# 6. Minimalist release: stage, commit, pull, push, tag and snapshot in
#     one call.
$ pixi run cgitSync freeze-release v1.1.0 "release v1.1.0"

# 7. Return to a frozen release at any later point.
$ pixi run cgitSync checkout v1.1.0 --ref-kind tag
```

The only `.cgs` idea this level relies on is the **name-matching auto-mount**: because CGSil1’s repository identifier’s last segment equals `project = "CGSil1"`, the root is inferred at `relative_path = "."` with no override written anywhere. That convenience is exactly what stops being safe once a project’s identifier and display name diverge — which is where Level 3a picks up.

6.2 Level 2 — cawaqs: the same minimal style, at scale

`cawaqs` scales Level 1’s minimal, no-override authoring style up to a real 19-repository build tree (root + 17 physics libraries across five GitLab groups + one required build-tooling repo), instead of a synthetic 4-repo one. Every entry’s on-disk layout already matches the default `relative_path = <repo_name>`, so not a single `relative_path` override is written anywhere in `examples/cawaqs.cgs` — only the shared branch-fallback convention. Full narrative in `tutorials/03_onboarding_a_real_build_tree.md`.

```

$ pixi run cgitSync validate examples/cawaqs.cgs
$ pixi run cgitSync bootstrap examples/cawaqs.cgs cawaqs-smoke-test
$ pixi run cgitSync view-tree --search-dir <path bootstrap printed>

# Move the whole 19-repo tree to a shared branch in one call, falling
# back to main per-repo wherever that branch does not exist.
$ pixi run cgitSync checkout my-feature-branch

```

Reaching READY here only replaces the *repository-fetching and branch-selection* half of `cawaqs`’s own `make_Cawaqs.sh/make_Cawaqs_from_branches.sh` scripts. `ComplexGitSync` does not compile anything: the remaining `make -f Makefile` build step is still the user’s job, and only succeeds from this tree with `LIB_HYDROSYSTEM_PATH` left unset or pointed at the bootstrapped root — the alternative shared, decoupled install layout those scripts also support is out of scope for `ComplexGitSync`’s containment model.

`discover` (Level 3b below) is not a route into `cawaqs.cgs`: the 17 library repositories never coexist inside one directory tree until *after* a `.cgs` already lists them, so there is no single checkout for a filesystem walk to scan. Authoring this one from documented developer knowledge, the way Level 3a does for `cawaqsviz`, is the only available route.

6.3 Level 3 — `cawaqsviz`: three ways to the same `.cgs`

`cawaqsviz` (<https://gitlab.com/cawaqs/gviz/cawaqsviz>) is a real GitLab project, nested three path segments deep, with two GitHub children that were, until recently, plain git submodules. One of those children, `HydrologicalTwinAlphaSeries`, holds a submodule of its own at `docs/hydrological_twin`: four repositories on two levels, which is why every command below uses `-recursive`. It has no `.cgs` of its own anywhere upstream. That makes it a good vehicle for showing all three ways `ComplexGitSync` can produce a working `.cgs` for a project it does not already control the source of, ordered here by how much the operator has to already know about the tree before running the command. Full narrative, as one combined ten-step procedure rather than three independent routes, in `tutorials/04_adopting_a_real_project.md`.

6.3.1 3a — Write it by hand

The most explicit, and most error-prone, route: read the project’s own `.gitmodules` and topology, then author `examples/cawaqsviz.cgs` directly. Its corrected content and the exact mistakes the first draft made (wrong nested-subgroup identifier, missing `relative_path` overrides) are already spelled out in §1 “Local Authoring Spec”; this level is about running it, not re-deriving it:

```

$ pixi run cgitSync validate examples/cawaqsviz.cgs
$ pixi run cgitSync bootstrap examples/cawaqsviz.cgs cawaqsviz-demo
$ pixi run cgitSync view-tree --search-dir <path bootstrap printed>

```

`bootstrap` is used instead of `initialise` here because, unlike `CGSi11` above, this example is not meant to have `ComplexGitSync` cloned inside the tree it manages — see `bootstrap` in §1 for why the two commands pick different default locations.

6.3.2 3b — Derive it with `discover` (autodiscover mode)

The opposite route: start from nothing but a checkout, and let `ComplexGitSync` read the filesystem instead of writing the `.cgs` by hand. This is the route to prefer whenever a checkout is already available, since it cannot repeat the hand-authoring mistakes of 3a — `relative_path` falls out of the walk directly instead of being typed.

```

# A plain clone leaves submodule paths empty; initialise them first so
# discover has something to find at those paths. --recursive reaches the
# second level too: without it, hydrological_twin stays an empty
# directory and discover cannot see a repository that is not there.
$ git clone https://gitlab.com/cawaqsviz/gviz/cawaqsviz cawaqsviz
$ cd cawaqsviz && git submodule update --init --recursive

# Dry run: report what discover sees, write nothing.
$ pixi run cgitsync discover . --max-depth 3

# Satisfied with the report: draft the .cgs.
$ pixi run cgitsync discover . --write cawaqsviz-discovered.cgs
$ pixi run cgitsync validate cawaqsviz-discovered.cgs

```

The draft reconstructs 3a's hand-corrected file field-for-field: the root at `relative_path = "."` and every other repository at its real submodule path (`external/HydrologicalTwinAlphaSeries`, `docs/CWV_user_guide`, `external/HydrologicalTwinAlphaSeries/docs/hydrological_twin`) rather than their bare repo names. That last path lies inside `HydrologicalTwinAlphaSeries`, so the report marks it *inside*: `external/HydrologicalTwinAlphaSeries` and the file is read back with `HydrologicalTwinAlphaSeries` as its parent. No repository here has a `.cgs` of its own, and `discover` writes no `nested_config` override for that: the default `"auto"` already resolves a repository with zero nested `*.cgs` matches cleanly. See `discover` in §1 for exactly what is and is not reported (no network calls, unresolvable remotes are warned about and left out, never guessed at).

6.3.3 3c — Migrate it with submodules import

The historical route: before either `.cgs` above existed, `cawaqsviz` tracked its children as real git submodules. This mode starts from that state and converts it, rather than describing a tree that is assumed to already be plain clones.

```

# Dry run against a real cawaqsviz checkout that still has submodules:
# reports name, path, URL, branch and declaring .gitmodules per
# submodule;
# changes nothing. --recursive reaches HydrologicalTwinAlphaSeries' own
# submodule; without it, only the root's two are converted.
$ pixi run cgitsync submodules report /path/to/cawaqsviz --recursive

# Convert: drop the gitlinks, retire .gitmodules, extend .gitignore.
$ pixi run cgitsync submodules import /path/to/cawaqsviz --recursive

# Review and commit the resulting working-tree changes as usual. Two
# repositories have changes here, one per converted level.
$ cd /path/to/cawaqsviz && git status
$ git commit -m "chore: retire git submodules in favour of
  ComplexGitSync"

```

This is the whole job: turning gitlinks into plain clones on disk. It does not also write a `.cgs` — `.gitmodules` never records the root's own identity, and a checkout worth converting already has a `.cgs` (3a) or can get one from `discover` (3b), run before or after `submodules import`. The full before/after picture (`index`, `.gitmodules`, `.gitignore`) and the automated test that proves it against fixture repositories are in `tutorials/04_adopting_a_real_project.md` §4 — this level is the condensed command sequence, not a restatement of that walkthrough. `submodules import` and `discover` answer different questions about the same tree: `discover` reports what is *checked out*, `submodules import` reads what git *declares* in `.gitmodules` and acts on it — a submodule path that was never initialised is invisible to 3b but not to 3c.

6.3.4 A shared caveat across all three

`ComplexGitSync` stores no credentials and has no private-repository authentication story (§1 “submodules report and submodules import” and “discover”). All three routes above assume every repository in the tree is reachable anonymously or through credentials the ambient environment (`ssh-agent`, an HTTPS credential helper) already holds.

7 Configuration and Environment

`ComplexGitSync` uses project files for topology and does not require global topology configuration. Snapshot discovery can use `CGSHOME`; when it is not set, the `initialise(.cgs)` output-path default is `CGSPATH=../..` (nested mode: `ComplexGitSync` cloned inside the tree it manages); later commands can also walk upward from the current directory to find `.cgitsync/state/`. `bootstrap(.cgs)` uses a different default instead — `CGSPATH=$HOME/.cgs/CGS<timestamp>/`, created automatically — since it is meant for a `ComplexGitSync` clone that is not nested inside any project tree; see `bootstrap` above. The logging and runtime-state subsystems use the user state directory by default:

- logs: `/.local/state/ComplexGitSync/logs/`
- latest runtime-state mapping: `/.local/state/ComplexGitSync/runtime/`

If `XDG_STATE_HOME` is set, these directories are rooted there instead.

8 Error Reference

ConfigValidationError Raised when `project` or `repos` is missing, a repository identifier is malformed, or a canonical field is invalid (for example an unknown `gitprovider`).

GitSyncError Raised when clone-time Git operations fail, for example because the remote branch cannot be resolved or the target directory is already occupied.

Exit code 1 General CLI failure (parse error, validation error).

Exit code 2 Command exists but is not yet implemented.

9 Troubleshooting

“gitprovider_url is required for custom provider” Add a `gitprovider_url` field to the offending `[[repos]]` entry in your `.cgs` file.

validate exits non-zero despite a correct-looking file Check that `project` is present and every `repos` entry uses `provider:owner/repository`. For advanced `[[repos]]` blocks without a repository shorthand, include `project_owner_name` and `project_name`.

Clone destination already exists and is not empty Move the existing directory aside yourself, or point `bootstrap` at a fresh `-cgs-path`.

No cloneable branch found for <repo> Confirm that the remote exposes either the repo’s `default_branch` or its `fallback_branch`.

PyYAML import error when calling from_yaml/to_yaml Add PyYAML in the Pixi environment: `pixi add pyyaml`.